

B28: SQL SuperHero Program : Session 26

1 message

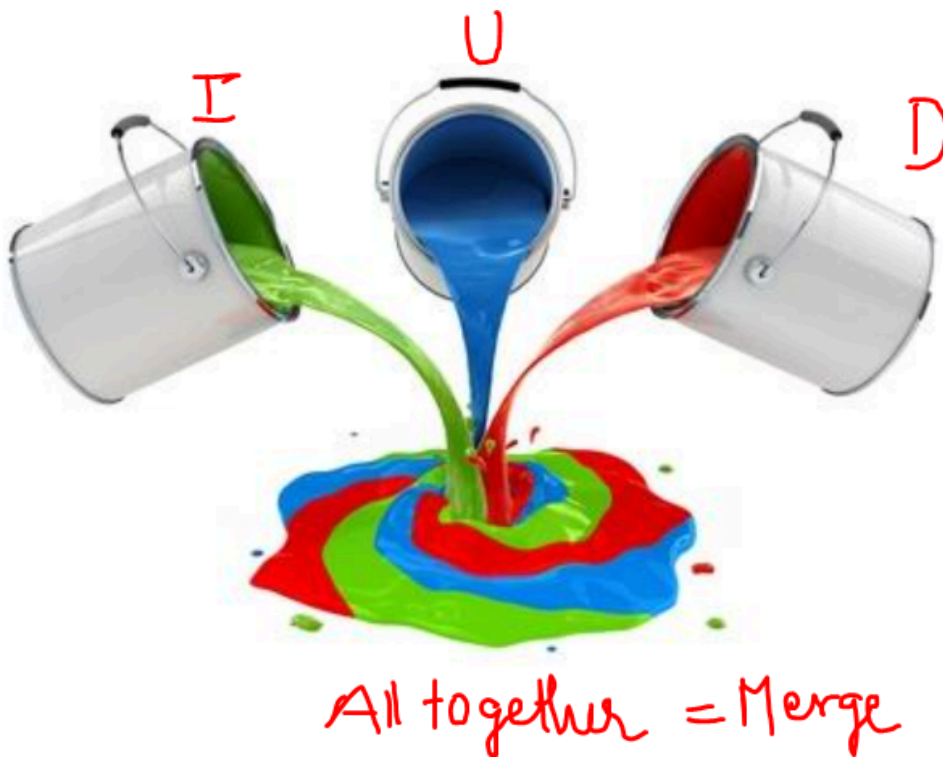
Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattekar1997@gmail.com

28 March 2025 at 02:05

Merge Statement

MERGE statement selects data from one or more source tables and updates or inserts it into a target table. The MERGE statement allows you to specify a condition to determine whether to update data from or insert data into the target table.

It's an alternative to a combination of INSERT and UPDATE and delete statements, and it's faster, and easier to code.



```
MERGE INTO target_table
USING source_table
ON search_condition
  WHEN MATCHED THEN
    UPDATE SET col1 = value1, col2 = value2,...
    WHERE <update_condition>
    [DELETE WHERE <delete_condition>]
  WHEN NOT MATCHED THEN
    INSERT (col1,col2,...)
    values(value1,value2,...)
    WHERE <insert_condition>;
```

First, specify the target table (target_table) which you want to update or insert into in the INTO clause.

Second, specify the source of data (source_table) to be updated or inserted in the USING clause.

Third, specify the search condition upon which the merge operation either updates or inserts in the ON clause.

For each row in the target table, Oracle evaluates the search condition:

If the result is true, then Oracle updates the row with the corresponding data from the source table.

In case the result is false for any rows, then Oracle inserts the corresponding row from the source table into the target table.

```
create table catalog1 (id number(3), item varchar2 (20), price number(6));  
insert into catalog1 values(1, 'laptop', 800);
```

```
insert into catalog1 values(2, 'iphone', 500);
```

```
insert into catalog1 values(3, 'camera', 700);
```

```
select * from catalog1;
```

Worksheet

Query Builder

1

2

```
select * from catalog1;
```

Query Result x

Query Result 1 x

Script Output x

Query x

SQL | All Rows Fetched: 3 in 0.562 seconds

	ID	ITEM	PRICE
1	1	laptop	800
2	2	iphone	500
3	3	camera	700

```
create table catalog2 (id number(3), item varchar2 (20), price number(6));
```

```
insert into catalog2 values(1, 'laptop', 899);
```

```
insert into catalog2 values(2, 'iphone', 599);
```

```
insert into catalog2 values(5, 'video camera', 799);
```

```
select * from catalog2;
```

Worksheet		Query Builder
3		<code>select * from catalog2;</code>
4		

Query Result x Query Result 1 x Script Output x Query R			
SQL All Rows Fetched: 3 in 0.546 seconds			
ID	ITEM	PRICE	
1	laptop	899	
2	iphone	599	
3	5video camera	799	

Merge all the new info into the original catalog1 table

```

MERGE INTO catalog1 s1
USING catalog2 s2
ON (s1.id = s2.id)
WHEN MATCHED THEN UPDATE SET s1.price = s2.price
WHEN NOT MATCHED THEN INSERT (id, item, price) values (s2.id, s2.item, s2.price);

```

5		
6		<code>MERGE INTO catalog1 s1</code>
7		<code>USING catalog2 s2</code>
8		<code>ON (s1.id = s2.id)</code>
9		<code>WHEN MATCHED THEN UPDATE SET s1.price = s2.price</code>
10		<code>WHEN NOT MATCHED THEN INSERT (id, item, price) values (s2.id, s2.item, s2.price);</code>

Query Result x Query Result 1 x Script Output x Query Result 2 x			
SQL All Rows Fetched: 4 in 0.577 seconds			
ID	ITEM	PRICE	
1	laptop	899	
2	iphone	599	
3	camera	700	
4	5video camera	799	

--> Merge will work on ON condition and check for both insert and update options. For laptop and iphone both match, so it updates the price
and for camera, no change and video camera is a new record in catalog2 table, and does not exist in catalog1 table so it is inserted.

```

5
6 MERGE INTO catalog1 s1
7 USING catalog2 s2
8 ON (s1.item = s2.item)
9 WHEN MATCHED THEN UPDATE SET s1.price = s2.price
10 WHEN NOT MATCHED THEN INSERT (id, item, price) values (s2.id, s2.item, s2.price);
11

```

ID	ITEM	PRICE
1	laptop	899
2	iphone	599
3	camera	700
4	video camera	799

--> Same On condition will work on Item also. It can work on any column, not only id.

```

CREATE TABLE members (
  member_id NUMBER PRIMARY KEY,
  first_name VARCHAR2(50) NOT NULL,
  last_name VARCHAR2(50) NOT NULL,
  rank VARCHAR2(20)
);

```

```

CREATE TABLE member_staging AS
SELECT * FROM members;

```

-- insert into members table

```

INSERT INTO members(member_id, first_name, last_name, rank) VALUES(1,'Abel','Wolf','Gold');
INSERT INTO members(member_id, first_name, last_name, rank) VALUES(2,'Clarita','Franco','Platinum');
INSERT INTO members(member_id, first_name, last_name, rank) VALUES(3,'Darryl','Giles','Silver');
INSERT INTO members(member_id, first_name, last_name, rank) VALUES(4,'Dorthea','Suarez','Silver');
INSERT INTO members(member_id, first_name, last_name, rank) VALUES(5,'Katrina','Wheeler','Silver');
INSERT INTO members(member_id, first_name, last_name, rank) VALUES(6,'Lilian','Garza','Silver');
INSERT INTO members(member_id, first_name, last_name, rank) VALUES(7,'Ossie','Summers','Gold');
INSERT INTO members(member_id, first_name, last_name, rank) VALUES(8,'Paige','Mcfarland','Platinum');
INSERT INTO members(member_id, first_name, last_name, rank) VALUES(9,'Ronna','Britt','Platinum');
INSERT INTO members(member_id, first_name, last_name, rank) VALUES(10,'Tressie','Short','Bronze');

```

-- insert into member_staging table

```

INSERT INTO member_staging(member_id, first_name, last_name, rank) VALUES(1,'Abel','Wolf','Silver');
INSERT INTO member_staging(member_id, first_name, last_name, rank) VALUES(2,'Clarita','Franco','Platinum');
INSERT INTO member_staging(member_id, first_name, last_name, rank) VALUES(3,'Darryl','Giles','Bronze');
INSERT INTO member_staging(member_id, first_name, last_name, rank) VALUES(4,'Dorthea','Gate','Gold');

```

```
INSERT INTO member_staging(member_id, first_name, last_name, rank) VALUES(5,'Katrina','Wheeler','Silver');
INSERT INTO member_staging(member_id, first_name, last_name, rank) VALUES(6,'Lilian','Stark','Silver');
```

When updating data from the members table to member_staging table, we should perform the following actions:

We update the rows with member id 1, 3, 4, and 6 because the rank or the last name of these members in these tables are different.

We insert the rows with member id 7 to 10 are because these rows exist in the members table but not in the member_staging table.

```
MERGE INTO member_staging x
USING (SELECT member_id, first_name, last_name, rank FROM members) y
ON (x.member_id = y.member_id)
WHEN MATCHED THEN
    UPDATE SET x.first_name = y.first_name,
              x.last_name = y.last_name,
              x.rank = y.rank
WHERE x.first_name <> y.first_name OR
      x.last_name <> y.last_name OR
      x.rank <> y.rank
WHEN NOT MATCHED THEN
    INSERT(x.member_id, x.first_name, x.last_name, x.rank)
    VALUES(y.member_id, y.first_name, y.last_name, y.rank);
```

The screenshot shows a SQL IDE with a MERGE statement and its results. Handwritten annotations in blue and red explain the query logic and highlight specific rows in the result set.

Handwritten Annotations:

- Target:** Points to the `member_staging` table in the MERGE statement.
- Query is source:** Points to the `(SELECT member_id, first_name, last_name, rank FROM members) y` subquery.
- Condition:** Points to the `ON (x.member_id = y.member_id)` clause.
- Insert:** Points to the `WHEN NOT MATCHED THEN INSERT` block.

SQL Statement:

```
MERGE INTO member_staging x
USING (SELECT member_id, first_name, last_name, rank FROM members) y
ON (x.member_id = y.member_id)
WHEN MATCHED THEN
    UPDATE SET x.first_name = y.first_name,
              x.last_name = y.last_name,
              x.rank = y.rank
WHERE x.first_name <> y.first_name OR
      x.last_name <> y.last_name OR
      x.rank <> y.rank
WHEN NOT MATCHED THEN
    INSERT(x.member_id, x.first_name, x.last_name, x.rank)
    VALUES(y.member_id, y.first_name, y.last_name, y.rank);
```

Query Result:

MEMBER_ID	FIRST_NAME	LAST_NAME	RANK
1	Abel	Wolf	Gold
2	Clarita	Franco	Platinum
3	Darryl	Giles	Silver
4	Dorthea	Suarez	Silver
5	Katrina	Wheeler	Silver
6	Lilian	Garza	Silver
7	Paige	Mcfarland	Platinum
8	Tressie	Short	Bronze
9	Ossie	Summers	Gold
10	Ronna	Britt	Platinum

```

5 MERGE INTO member_staging x
6 USING (SELECT member_id, first_name, last_name, rank FROM members) y
7 ON (x.member_id = y.member_id)
8 WHEN MATCHED THEN
9     UPDATE SET x.first_name = y.first_name,
10                x.last_name = y.last_name,
11                x.rank = y.rank
12 WHEN NOT MATCHED THEN
13     INSERT(x.member_id, x.first_name, x.last_name, x.rank)
14     VALUES(y.member_id, y.first_name, y.last_name, y.rank);
15

```

Query Result x | Query Result 1 x | Script Output x | Query Result 2 x

Task completed in 0.54 seconds

10 rows merged.

→ All 10 merged

--> In above example, we are not using where clause while updating, so it does not check any thing and where id is matches as per ON condition, it straight forward go and update. so 6 records are updated and 4 new are inserted, so total 10.

So, you can see that conditional merge is done.

Example with DELETE MERGE:

```

create table bricks_for_sale (
  colour varchar2(10),
  shape  varchar2(10),
  price  number(10, 2)
);

```

```

create table purchased_bricks (
  colour varchar2(10),
  shape  varchar2(10),
  price  number(10, 2)
);

```

```

insert into bricks_for_sale values ( 'red', 'cube', 4.95 );
insert into bricks_for_sale values ( 'blue', 'cube', 7.75 );
insert into bricks_for_sale values ( 'blue', 'pyramid', 9.99 );

```

No records in table purchased_bricks. lets do merge:

```

merge into purchased_bricks pb
using bricks_for_sale bfs
on ( pb.colour = bfs.colour and pb.shape = bfs.shape )
when not matched then

```

```
insert ( pb.colour, pb.shape, pb.price )
values ( bfs.colour, bfs.shape, bfs.price )
when matched then
update set pb.price = bfs.price;
```

→ Merge Performed

```
1 merge into purchased_bricks pb
2 using bricks_for_sale bfs
3 on ( pb.colour = bfs.colour and pb.shape = bfs.shape )
4 when not matched then
5   insert ( pb.colour, pb.shape, pb.price )
6   values ( bfs.colour, bfs.shape, bfs.price )
7 when matched then
8   update set pb.price = bfs.price;
9
10 select * from purchased_bricks;
```

COLOUR	SHAPE	PRICE
red	cube	4.95
blue	cube	7.75
blue	pyramid	9.99

Download CSV
3 rows selected.

← All Records

Lets use DELETE

Update 1 entry

1. update bricks_for_sale set price = 0.99;
2. insert into bricks_for_sale values ('red', 'pyramid', 5.99);
3. insert into purchased_bricks values ('green', 'cube', 9.95);

Now records in bricks_for_sale have changed the price. And 1 new record inserted in both tables.

```
1 select * from purchased_bricks;
```

target table

COLOUR	SHAPE	PRICE
red	cube	4.95
blue	cube	7.75
blue	pyramid	9.99
green	cube	9.95

[Download CSV](#)
4 rows selected.

```
1 select * from bricks_for_sale;
```

source table

COLOUR	SHAPE	PRICE
red	cube	.99
blue	cube	.99
blue	pyramid	.99
red	pyramid	5.99

[Download CSV](#)
4 rows selected.

We can see the records in source and target both. Let see the below merge statement :


```

1 merge into purchased_bricks pb
2 using bricks_for_sale bfs
3 on ( pb.colour = bfs.colour and pb.shape = bfs.shape )
4 when not matched then
5   insert ( pb.colour, pb.shape, pb.price )
6   values ( bfs.colour, bfs.shape, bfs.price )
7 when matched then
8   update set pb.price = bfs.price
9   delete where pb.colour = 'red';
10
11 select * from purchased_bricks
12
13
14
15

```

when match then delete

COLOUR	SHAPE	PRICE
blue	cube	.99
blue	pyramid	.99
green	cube	9.95

color: 'red' is deleted

Download CSV
3 rows selected.

--> Delete is used only with "When Matched". Delete can not be used with "When NOT Matched"

- We can do only when matched in merge
- We can do only when not matched in merge
- They can be in any sequence, either matched first or not matched first
- We can not do delete alone

```

8 MERGE INTO purchased_bricks PB
9 USING bricks_for_sale BFS
10 ON (PB.COLOUR = BFS.COLOUR and PB.SHAPE = BFS.SHAPE)
11 WHEN MATCHED THEN
12 delete where PB.price = 0.99
13
14

```

→ Delete can not use alone.

it must accompany with update + delete

Query Result x | Query Result 1 x | Script Output x | Query Result 2 x

Task completed in 0.044 seconds

```

MERGE INTO purchased_bricks PB
USING bricks_for_sale BFS
ON (PB.COLOUR = BFS.COLOUR and PB.SHAPE = BFS.SHAPE)
WHEN MATCHED THEN
delete where PB.price = 0.99
Error at Command Line : 12 Column : 1
Error report -
SQL Error: ORA-00905: missing keyword
00905. 00000 - "missing keyword"
*Cause:
*Action:

```

=====

=====

Merge Restrictions:

There are a couple of things you need to watch for in the when matched clause. You can only update:

1. columns not in the join clause
2. each row once (if source have duplicate, can not do merge)

```

1 merge into purchased_bricks pb
2 using bricks_for_sale bfs
3 on ( pb.colour = bfs.colour and pb.shape = bfs.shape )
4 when not matched then
5   insert ( pb.colour, pb.shape, pb.price )
6   values ( bfs.colour, bfs.shape, bfs.price )
7 when matched then
8   update set pb.colour = bfs.colour, pb.shape = bfs.shape;
9
10
11
12

```

on condition → try to update

ORA-38104: Columns referenced in the ON Clause cannot be updated: "PB"."COLOUR"

```
8 MERGE INTO purchased_bricks PB
9 USING bricks_for_sale BFS
10 ON (PB.COLOUR = BFS.COLOUR and PB.SHAPE = BFS.SHAPE)
11 WHEN MATCHED THEN
12 Update set pb.colour = 'green'
13
14
```

we used colour column in ON condition

X not allowed

Query Result x | Query Result 1 x | Script Output x | Query Result 2 x | Script Output 1 x

Task completed in 0.018 seconds

Error starting at line : 8 in command -

```
MERGE INTO purchased_bricks PB
USING bricks_for_sale BFS
ON (PB.COLOUR = BFS.COLOUR and PB.SHAPE = BFS.SHAPE)
WHEN MATCHED THEN
Update set pb.colour = 'green'
```

Error at Command Line : 10 Column : 5

Error report -

SQL Error: ORA-38104: Columns referenced in the ON Clause cannot be updated: "PB"."COLOUR"

see this

=====

=====

SQL Worksheet

```
1 merge into purchased_bricks pb
2 using bricks_for_sale bfs
3 on ( pb.colour = bfs.colour )
4 when not matched then
5 insert ( pb.colour, pb.shape, pb.price )
6 values ( bfs.colour, bfs.shape, bfs.price )
7 when matched then
8 update set pb.price = bfs.price;
```

multiple record

ORA-30926: unable to get a stable set of rows in the source tables ORA-06512: at "SYS.DBMS_SQL", line 1721

If My source (using clause) has duplicate value against the ON condition, we can not use merge.

Worksheet Query Builder

```

1 select * from bricks_for_sale;
2
3
4 MERGE INTO purchased_bricks PB
5 USING bricks_for_sale BFS
6 ON (PB.COLOUR = BFS.COLOUR)
7 WHEN MATCHED THEN
8 Update set pb.price = 50

```

my source

Data

2 records for BLUE

*Based upon ON condition
↓
Duplicate in source*

	COLOUR	SHAPE	PRICE
1	red	cube	4.95
2	blue	cube	7.75
3	blue	pyramid	9.99

Script Output x Query Result x

SQL | All Rows Fetched: 3 in 0.003 seconds

Now, if we see, we have 2 rows in source for colour = BLUE and this is our ON condition, so merge will not work here.

```

3
4 MERGE INTO purchased_bricks PB
5 USING bricks_for_sale BFS
6 ON (PB.COLOUR = BFS.COLOUR)
7 WHEN MATCHED THEN
8 Update set pb.price = 50

```

Script Output x Query Result x

Task completed in 0.019 seconds

Error starting at line : 4 in command -
 MERGE INTO purchased_bricks PB
 USING bricks_for_sale BFS
 ON (PB.COLOUR = BFS.COLOUR)
 WHEN MATCHED THEN
 Update set pb.price = 50
 Error report -
 ORA-30926: unable to get a stable set of rows in the source tables

This will be the error.

=====

=====

✓ **Target**

→ **Source**

```

1: MERGE INTO MainTable PR
2:   USING IncrementalTable PRA
3:   ON (PR.contract = PRA.contract
4:      and PRA.memberType = 'Parent' ) ] — condition
5:   WHEN MATCHED THEN
6:     UPDATE SET PR.address1 = PRA.address1,
7:               PR.city = PRA.city,
8:               PR.state = PRA.sate,
9:               PR.zipCode = PRA.zipCode,
10:              PR.countryCode = PRA.countryCode
11: WHERE address1 IS NULL
12:   AND PR.memberType <> 'Parent'
13: ;

```

Error getting : **ORA-30926: unable to get a stable set of rows in the source tables**

Why this error come? because in source table I have multiple rows against the on condition. Source table is : Incremental Table and on condition is based on contact and membertype column. let me cross check:

```

1: select contract,
2:        memberType,
3:        count(*)
4:   from IncrementalTable
5:  group by contract,
6:        memberType
7: having count(*) > 1
8: ;
9:
10: CONTRACT          MEMBERTYPE    COUNT (*)
11: -----
12: 1119839490         PARENT           2
13: 7271122516         PARENT           2
14: 1004798721         PARENT           2

```

How to solve it. try something to get unique data. like

```

USING (SELECT distinct contract, address1, city, state, zipCode, countryCode
FROM IncrementalTable
WHERE memberType = 'Parent')

```

=====

=====

Performance Comparison Merge Vs Insert/Update

MERGE is useful for combining larger source and target datasets. Never say in interview that merge is faster or update is faster. Performance of the MERGE statement and the UPDATE statement in SQL involves various factors such as the specific use case, the size of the data, indexing, and the database management system being used.

User case scenario:

MERGE can reduce the number of round-trips to the database since it combines multiple operations into one. When combining UPDATE and INSERT operations, you might need to execute two separate statements, resulting in more database round-trips. It's efficient for scenarios where you need to perform multiple actions on the same data based on a condition. It's useful for synchronizing data or upserting (update or insert) records.

The MERGE statement typically involves more overhead because it needs to evaluate multiple conditions, such as matching and non-matching rows, and perform both update and insert operations. This can make it slower for simple update tasks.

Data Volume Scenario :

MERGE: If you have a large volume of data or complex conditions, MERGE might be less efficient due to the additional processing involved.

UPDATE: UPDATE statements can be more efficient for updating large datasets when there are no inserts involved.

Indexing Scenario: (We will learn index later)

MERGE: The performance of the MERGE statement can be greatly affected by indexing. Make sure that indexes are appropriately set up to optimize both the matching and non-matching conditions.

UPDATE: Like MERGE, indexing is crucial for UPDATE statements. Ensure that columns used in the WHERE clause are indexed for faster lookups.

Maintenance Scenario:

MERGE: MERGE statements can be more complex to write and maintain due to their conditional nature. They require careful consideration of the logic to avoid unintended updates or inserts.

UPDATE: UPDATE statements tend to be simpler and easier to maintain, especially for straightforward updates.

It's important to benchmark and profile both approaches using real data and monitor query execution plans to determine which approach performs better in your specific database environment.

We can not say by default which one is faster.

Merge Vs Update :

1. Never give answer that which one is better or faster. It all depends on data, complexity and use cases.
2. Merge will be more secure and a better way of handling complex situations.
3. Update seems faster if we do straightforward update the data and merge will take more CPU because of its transaction handling feature.
4. Merge gives option to do multiple dml together and even it will do delete from target table. so merge comes with extra feature
5. We should use both depends on our need .

Not in Vs Not exists :

1. Both have their use case and we can not benchmark as a general answer.
2. If my inner query have any null data, NOT IN will not work. It will be unable to handle null values.
3. IN is an extension of = and null data undefined and unknown, we can not compare. because of that NOT IN unable to compare NULL data.
4. Not exists is boolean , it just check yes or no.
5. If my inner query has huge volume of data then exists will faster .
6. If my inner query has less data then IN will faster